

Kiến trúc SOA & Microservices

Đặng Ngọc Hùng

Mục lục

12. Triển khai & DevOps	4
12.1. Thách thức Triển khai Microservices	5
12.1.1. Vấn đề: từ “deploy một file WAR” đến “deploy N services đồng bộ”	5
12.1.2. Từ thủ công đến tự động hóa — Tư duy DevOps	5
12.2. Containerization với Docker	6
12.2.1. Container — Cô lập gọn nhẹ	6
12.2.2. Các mô hình triển khai — Từ đặc thù ngôn ngữ đến Serverless	7
12.2.3. Dockerfile — Định nghĩa Container Image	8
12.3. Docker Compose — Điều phối trên một máy	8
12.3.1. Docker Compose — Khai báo nhiều service	9
12.3.2. Docker Compose — Điểm mạnh và giới hạn	10
12.4. CI/CD Pipeline cho Microservices	11
12.4.1. Kiến trúc CI/CD cho Microservices	12
12.4.2. Mono-repo vs Poly-repo — Ảnh hưởng đến CI/CD	12
12.4.3. Tối ưu hóa Pipeline	12
12.4.4. Ví dụ Thực tế: CI/CD pipeline cho KBM	13
12.4.5. Quản lý Secrets	16
12.4.6. Database Migration trong CI/CD Pipeline	16
12.5. Các chiến lược triển khai	17
12.5.1. Rolling Update	18
12.5.2. Blue/Green Deployment	18
12.5.3. Canary Release	19
12.5.4. Shadow Deployment	19
12.5.5. So sánh	20
12.6. Infrastructure as Code (IaC)	20
12.6.1. Vấn đề: “Ai cấu hình server? Config ở đâu?”	20
12.6.2. IaC — Hạ tầng như Code	20
12.6.3. Docker Compose như một dạng IaC	21
12.6.4. Kubernetes — Container Orchestration cho Production	22
12.6.5. Triển khai Serverless — Khi nào phù hợp?	26
12.6.6. Sidecar Pattern và Service Mesh	27
12.7. Case Study: KBM	28
12.7.1. Hiện trạng	28
12.7.2. Phân tích theo mức độ trưởng thành triển khai	29
12.7.3. Phân tích bối cảnh nghiệp vụ	30
12.8. Tổng kết	31
12.9. Đọc thêm	32
Tài liệu tham khảo	33

12

CHƯƠNG 12

Triển khai & DevOps

“If deploying is painful, deploy more frequently. Automation turns a dreaded chore into a non-event.”

— Jez Humble & David Farley, *Continuous Delivery* (2010)

MỤC TIÊU HỌC TẬP

- Hiểu vì sao triển khai microservices phức tạp hơn monolith và cần **tự động hóa**
- Nắm vững **container hóa** (containerization) với Docker — từ Dockerfile đến image registry
- Sử dụng **Docker Compose** để điều phối việc triển khai nhiều service trên một máy
- Thiết kế **CI/CD pipeline** cho microservices — build, test, deploy tự động
- So sánh các **chiến lược triển khai** (deployment strategies) Rolling, Blue/Green, Canary và Shadow
- Hiểu **Infrastructure as Code** (IaC) và vai trò trong quản lý hạ tầng
- Phân tích kiến trúc triển khai của KBM và đề xuất cải thiện

12.1. Thách thức Triển khai Microservices

Triển khai monolith chỉ cần copy một artifact duy nhất, nhưng với microservices, đó là bài toán điều phối hàng chục dịch vụ đồng thời. Sự phức tạp này bắt buộc phải có các luồng tự động hóa chuyên biệt.

12.1.1. Vấn đề: từ “deploy một file WAR” đến “deploy N services đồng bộ”

Trong monolith, triển khai nghĩa là: build một artifact (JAR/WAR), copy lên server, restart. Đối với kiến trúc nguyên khối, toàn bộ quy trình từ biên dịch, kiểm thử cho đến triển khai chỉ xoay quanh một tệp tin duy nhất trong một lần thực thi.

Trong microservices, “deploy” mang một ý nghĩa khác hẳn so với monolith. Thay vì đóng gói một ứng dụng duy nhất và đưa lên một server, hệ thống phải quản lý hàng chục, thậm chí hàng trăm artifact riêng lẻ trên nhiều server. Cơ sở dữ liệu cũng vậy: monolith chỉ cần một luồng migration thống nhất, còn mỗi microservice lại sở hữu schema riêng và phải nâng cấp độc lập nhưng vẫn tương thích với nhau.

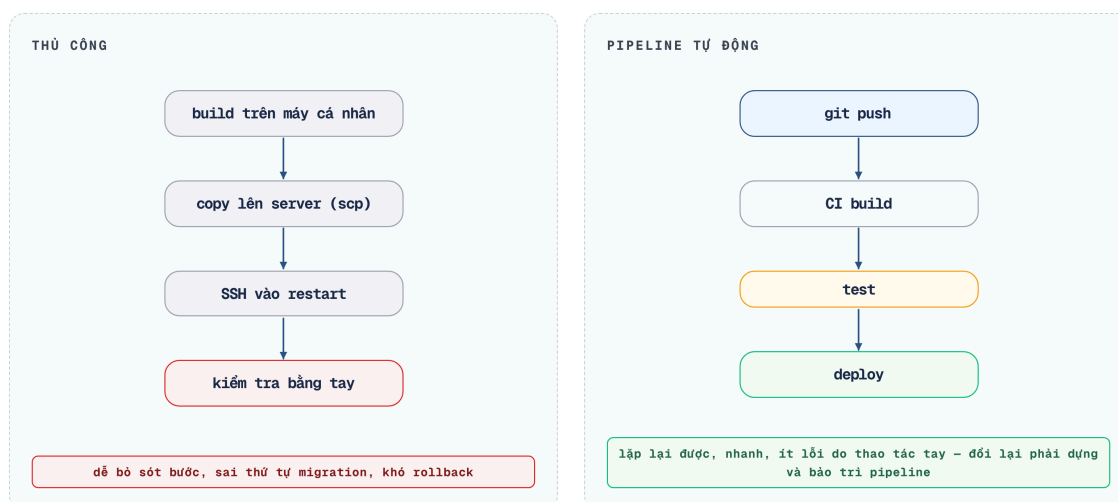
Quá trình khởi động lại hệ thống cũng phức tạp không kém; việc khởi động nhiều tiến trình trong microservices đòi hỏi một trình tự nghiêm ngặt để tránh lỗi phụ thuộc chéo. Khi có sự cố cần quay lui (rollback), việc đưa nhiều services về các phiên bản tương thích phức tạp hơn hẳn so với việc lùi một bước của monolith. Cuối cùng, việc kiểm thử cũng không còn đơn giản, vì một service thường cần các service liên quan chạy cùng để đảm bảo tính chính xác trong môi trường tích hợp. Newman trong [4a, Ch.6] nhấn mạnh: khả năng **triển khai độc lập** (independent deployment) là lợi ích cốt lõi của microservices — nhưng cũng là thách thức lớn. Nếu không tự động hóa, việc triển khai thủ công từ 7 dịch vụ trở lên sẽ trở thành “nỗi ám ảnh chiều thứ Sáu”.

! Triển khai độc lập

Khả năng deploy từng service độc lập là lý do cốt lõi để chọn microservices — nhưng không có CI/CD, kỷ luật quản lý phiên bản và hạ tầng bất biến (immutable infrastructure), lợi ích này biến thành gánh nặng. Mỗi lần phát hành thủ công là rủi ro: bỏ sót bước, sai thứ tự migration, thiếu kế hoạch rollback. Tự động hóa DevOps không phải tùy chọn trong microservices — đó là **điều kiện bắt buộc** để lợi ích độc lập trở thành thực tế.

12.1.2. Từ thủ công đến tự động hóa — Tư duy DevOps

Theo Mitra [3, Ch.6], ba nguyên tắc cốt lõi của DevOps cho microservices bao gồm: **Immutable Infrastructure** (Hạ tầng bất biến), yêu cầu tuyệt đối không can thiệp trực tiếp lên máy chủ đang chạy mà phải khởi tạo phiên bản mới và loại bỏ phiên bản cũ, nhằm đảm bảo khả năng tái tạo và triệt tiêu sự sai lệch cấu hình (configuration drift). Kế tiếp là **Infrastructure as Code** (IaC), biến mọi thao tác cấu hình server, network, hay database thành các đoạn code được lưu trữ trên Git, giúp dễ dàng review và rollback. Cuối cùng, **Continuous Delivery** (Phân phối liên tục) đảm bảo codebase luôn ở trạng thái sẵn sàng tung ra thị trường chỉ với một nút bấm, qua đó chia nhỏ rủi ro và tăng tốc vòng lặp phản hồi.



Hình 12.1: Triển khai thủ công vs tự động

Mô hình triển khai tự động giúp loại bỏ các bước thủ công dễ sinh lỗi và giảm rủi ro khi phát hành phần mềm. Quá trình phân phối tự động phản ánh triết lý vận hành cốt lõi của DevOps: biến khó khăn thành động lực tự động hóa (if it hurts, do it more often).

! If It Hurts, Do It More Often

“If deploying is painful, deploy more frequently. The pain is information — it tells you what to automate.”

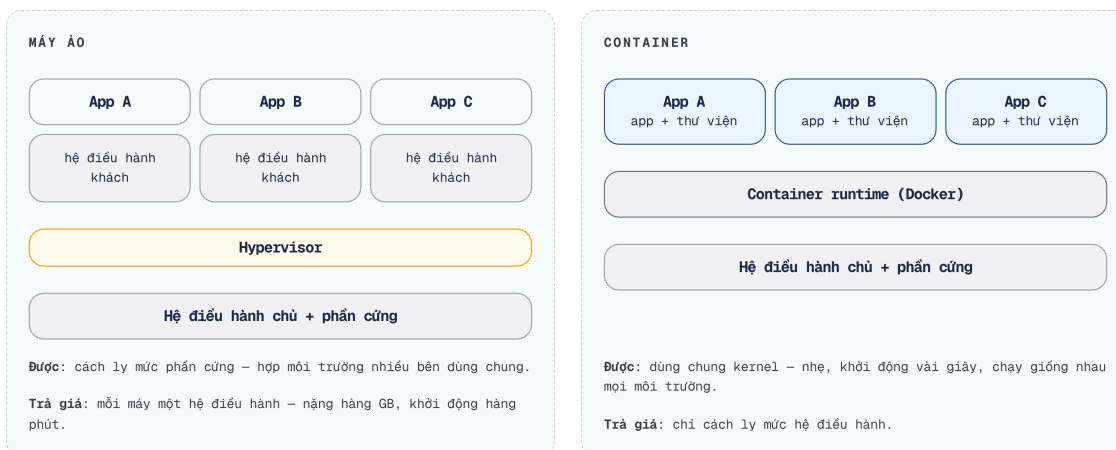
— Jez Humble & David Farley, *Continuous Delivery* (trích dẫn bởi Mitra [3, Ch.6])

12.2. Containerization với Docker

Vấn đề: “Works on my machine” Sự không nhất quán giữa môi trường phát triển (máy cá nhân) và môi trường production thường xuyên gây ra lỗi ‘Works on my machine’. Container hóa giải quyết bài toán này.

12.2.1. Container — Cô lập gọn nhẹ

Container (Docker) đóng gói ứng dụng cùng với các thư viện phụ thuộc (dependencies) và môi trường thực thi (runtime) thành một **image** — chạy giống nhau trên mọi môi trường. Khác với máy ảo (Virtual Machine): container chia sẻ kernel với hệ điều hành máy chủ giúp tối ưu hóa đáng kể dung lượng, khởi động trong giây thay vì phút.



Hình 12.2: So sánh kiến trúc máy ảo và container

Sự khác biệt về kiến trúc cốt lõi giữa hai công nghệ này dẫn đến những khác biệt đáng kể về hiệu năng và cách thức vận hành. Về thời gian khởi động, máy ảo thường mất hàng phút do phải nạp lại toàn bộ hệ điều hành khách, trong khi các container chỉ mất vài giây vì được dùng chung kernel với máy chủ. Về tài nguyên, một máy ảo đòi hỏi dung lượng lưu trữ lên đến hàng gigabyte, trong khi container tối ưu hóa đáng kể tài nguyên do chỉ bao gồm ứng dụng và các thư viện cần thiết. Sự đánh đổi này thể hiện rõ qua mức độ cách ly; máy ảo bảo mật ở mức phần cứng cực tốt cho môi trường chia sẻ đa khách, còn container chỉ cách ly ở mức hệ điều hành.

🔍 VM vs Container / So sánh kiến trúc

Để dễ hình dung sự khác biệt về bản chất:

Virtual Machine (VM) – Giống như việc thuê một căn hộ độc lập trong một tòa nhà chung cư. Người dùng có phòng khách, bếp, vệ sinh riêng hoàn toàn (Hệ điều hành riêng). Rất riêng tư và an toàn, nhưng tiêu tốn nhiều không gian và đòi hỏi thời gian thiết lập ban đầu đáng kể.

Container – Giống như việc thuê một phòng trong một không gian làm việc chung (Co-working space). Người dùng có không gian làm việc riêng (User space), nhưng dùng chung hệ thống điện, nước, internet, lễ tân của tòa nhà (Kernel). Tối ưu hóa tài nguyên và cho phép đưa vào vận hành ngay lập tức.

Richardson trong [2a, Ch.12] mô tả 5 mô hình triển khai (deployment patterns), trong đó **Service as Container** là phổ biến nhất cho microservices – cân bằng giữa mức độ cách ly, tốc độ khởi tạo và hiệu suất sử dụng tài nguyên.

12.2.2. Các mô hình triển khai – Từ đặc thù ngôn ngữ đến Serverless

Theo Richardson [2a, Ch.12], có năm mô hình triển khai chính trải dài từ sự đơn giản đến những kiến trúc phức tạp. Cách tiếp cận nguyên sơ nhất là triển khai ứng dụng bằng ngôn ngữ đặc thù trực tiếp lên application server; dù đơn giản nhưng lại tiềm ẩn nguy cơ cao xảy ra lỗi xung đột thư viện do thiếu tính cách ly. Kế tiếp, việc đặt mỗi dịch vụ lên một máy ảo riêng giải quyết được bài toán cách ly, nhưng lại phải trả giá bằng lượng tài nguyên khổng lồ và thời gian khởi tạo rất chậm. Mô hình mỗi dịch vụ một container (như Docker) cân bằng được tính mỏng nhẹ, khả năng di động và sự độc lập, dù vẫn cần công cụ điều phối ở quy mô lớn. Khi cần tối giản vận hành, kiến trúc

Serverless giao toàn bộ hạ tầng cho đám mây để đạt khả năng thu phóng tức thì, chấp nhận độ trễ khởi động và sự phụ thuộc nhà cung cấp. Cuối cùng, với các hệ thống đồ sộ, Kubernetes là nền tảng điều phối và tự phục hồi tiêu chuẩn, dù đòi hỏi nhiều công sức tiếp cận ban đầu.

12.2.3. Dockerfile — Định nghĩa Container Image

Dockerfile mô tả cách build container image cho một service. Mẫu phổ biến cho Spring Boot microservice:

```
# Multi-stage build — tách build environment khỏi runtime
FROM eclipse-temurin:21-jdk AS build
WORKDIR /app
COPY pom.xml mvnw ./
COPY .mvn .mvn
RUN ./mvnw dependency:go-offline -B      # Cache dependencies layer
COPY src src
RUN ./mvnw package -DskipTests          # Build JAR

# Runtime image nhỏ hơn JDK
FROM eclipse-temurin:21-jre
WORKDIR /app

# Tạo non-root user (giải pháp khuyến nghị về bảo mật)
RUN useradd -m appuser
USER appuser

COPY --from=build /app/target/*.jar app.jar
EXPOSE 8080
# Khởi động ứng dụng (Spring Boot tự quản lý Graceful Shutdown qua cấu hình)
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Listing 12.1: Dockerfile sử dụng Multi-stage build

Hai kỹ thuật quan trọng:

Sử dụng đa giai đoạn (Multi-stage build) – giai đoạn 1 (JDK) biên dịch tệp JAR, giai đoạn 2 (JRE) chỉ chứa môi trường thực thi, giúp giảm đáng kể dung lượng image

Lưu trữ lớp (Layer caching) – thực hiện sao chép tệp cấu hình trước mã nguồn (`COPY pom.xml` trước `COPY src`), giúp các thư viện phụ thuộc chỉ được tải lại khi cấu hình thay đổi, thay vì mỗi lần cập nhật mã

! Mỗi service một container

“Mỗi microservice chạy trong container riêng. Không đóng gói nhiều dịch vụ vào một container – mất tính cô lập, khó mở rộng, khó gỡ lỗi.”
– Nguyên tắc containerization (Richardson [2a, Ch.12], Newman [4a, Ch.6])

12.3. Docker Compose — Điều phối trên một máy

Vấn đề: khởi chạy thủ công nhiều dịch vụ, cơ sở dữ liệu và Kafka? Nền tảng KBM yêu cầu khởi động nhiều dịch vụ, cơ sở dữ liệu và message broker theo đúng thứ tự. Việc chạy thủ công từng thành phần làm tốn thời gian và dễ dẫn đến sai lệch cấu hình mạng lưới.

```
# SAI: khởi động thủ công bằng 10+ lệnh docker run, đúng thứ tự, đúng network, đúng
env vars
docker run -d postgres:15 ...
docker run -d zookeeper:3.8 ...
docker run -d kafka:3.5 ...
docker run -d eureka-server ...
docker run -d kbm-gateway --eureka-url=... ...
docker run -d kbm-core --db-url=... --kafka=... ...
# ... còn 5 services nữa
```

Listing 12.2: Khởi động thủ công các services (Anti-pattern)

12.3.1. Docker Compose — Khai báo nhiều service

Docker Compose định nghĩa **toàn bộ stack** trong một file YAML — services, networks, volumes, dependencies. Một lệnh `docker compose up` khởi động toàn bộ hệ thống.

Cấu trúc Docker Compose cho KBM LMS chính (đơn giản hóa):

```
# docker-compose.yml — KBM LMS chính
services:
  # --- Infrastructure ---
  postgres:
    image: postgres:15
    restart: unless-stopped
    environment:
      POSTGRES_DB: app_db
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 10s
      timeout: 5s
      retries: 5

  # Kafka chạy KRaft mode (production-ready từ 3.3, mặc định từ 4.0): broker tự quản
  # metadata,
  # không cần Zookeeper. Cụm dựa trên Zookeeper nay là legacy (gỡ bỏ ở Kafka 4.0).
  kafka:
    image: confluentinc/cp-kafka:7.5.0
    restart: unless-stopped
    environment:
      CLUSTER_ID: "Mku30EVBNTcwNTJENDM2Qk" # bắt buộc ở KRaft: format volume
      metadata
      KAFKA_PROCESS_ROLES: "broker,controller"
      KAFKA_NODE_ID: 1
      KAFKA_CONTROLLER_QUORUM_VOTERS: "1@kafka:9093"
      KAFKA_LISTENERS: "PLAINTEXT://:9092,CONTROLLER://:9093"
      # Listener tên CONTROLLER không trùng tên security protocol nào, nên phải
      # khai báo map tường minh; thiếu nó broker abort ngay khi validate config.
      KAFKA_LISTENER_SECURITY_PROTOCOL_MAP:
        "PLAINTEXT:PLAINTEXT,CONTROLLER:PLAINTEXT"
      KAFKA_CONTROLLER_LISTENER_NAMES: "CONTROLLER"
      KAFKA_INTER_BROKER_LISTENER_NAME: "PLAINTEXT"
      # Địa chỉ client nhận về sau bootstrap — thiếu thì service không kết nối được
      KAFKA_ADVERTISED_LISTENERS: "PLAINTEXT://kafka:9092"
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1 # single-node
    healthcheck:
```

```

    test: ["CMD", "kafka-cluster", "cluster-id", "--bootstrap-server",
"localhost:9092"]
    interval: 10s
    timeout: 5s
    retries: 5

# --- Platform Services ---
registry:
  image: ghcr.io/kbm/registry
  ports: ["9000:9000"]

kbm-gateway:
  image: ghcr.io/kbm/kbm-gateway
  ports: ["9001:9001"]
  depends_on: [registry]

# --- Application Services ---
kbm-core:
  image: ghcr.io/kbm/kbm-core:${IMAGE_TAG:-latest} # nhận tag từ pipeline; thiếu
biến này thì luôn kéo latest
  restart: unless-stopped
  depends_on:
    postgres:
      condition: service_healthy
    kafka:
      condition: service_healthy
    registry:
      condition: service_started
  environment:
    SPRING_DATASOURCE_URL: jdbc:postgresql://postgres:5432/app_db
  deploy:
    resources:
      limits:
        cpus: '1.0'
        memory: 512M

kbm-judge-sql:
  image: ghcr.io/kbm/kbm-judge-sql
  depends_on:
    kafka:
      condition: service_healthy
    registry:
      condition: service_started

volumes:
  pgdata:

```

Listing 12.3: Cấu hình Docker Compose cho KBM LMS chính

12.3.2. Docker Compose — Điểm mạnh và giới hạn

Đánh giá điểm mạnh và giới hạn của Docker Compose:

Về tính khai báo (Declarative) – Docker Compose định nghĩa toàn bộ stack trong một file cấu hình duy nhất, mang lại sự tiện lợi trong việc hình dung tổng thể hệ thống. Tuy nhiên, nền tảng này có nhược điểm lớn là **chỉ chạy được trên một máy duy nhất (single host)**.

Về tính tái tạo (Reproducible) – Chỉ với lệnh `docker compose up` là chúng ta có thể tái thiết lập kết quả một cách nhất quán ở mọi môi trường. Bù lại, công cụ này **không hỗ trợ tính năng**

tự phục hồi (self-healing). Nếu một container bị sập giữa chừng, nó sẽ không tự động được khôi phục trừ khi có các cài đặt phụ trợ cơ bản.

Về quản lý phụ thuộc (Dependencies) – Chỉ thị `depends_on` cho phép kiểm soát thứ tự khởi động và — khi kết hợp `condition: service_healthy` — đợi health check của container phụ thuộc thành công trước khi bật container kế tiếp. Tuy nhiên, health check thành công chỉ có nghĩa lệnh kiểm tra (ví dụ `pg_isready`) thành công; nó *không đảm bảo* ứng dụng bên trong đã hoàn tất mọi bước khởi tạo và sẵn sàng xử lý request nghiệp vụ. Do đó, ứng dụng vẫn cần cơ chế retry/backoff riêng khi kết nối tới các phụ thuộc. Ngoài ra, kiến trúc định tuyến của Compose **không có khả năng load balancing nội tại**, đòi hỏi người vận hành phải tự thiết lập một hệ thống reverse proxy bổ sung.

Về khả năng cô lập mạng (Isolated networking) – Tính năng mạng nội bộ cho phép các dịch vụ kết nối với nhau thông qua tên miền dịch vụ một cách cô lập. Điểm yếu là mạng lưới này **chưa đạt tiêu chuẩn production-grade**, thiếu các cơ chế như smart health check và rolling update.

Docker Compose **phù hợp cho** môi trường phát triển, staging, môi trường kiểm thử CI/CD, và **hệ thống production quy mô nhỏ-trung** (như KBM LMS chính). Khi có nhu cầu mở rộng quy mô trên nhiều máy chủ, tự phục hồi và cập nhật không gián đoạn, hệ thống cần đến **Kubernetes** hoặc một nền tảng container orchestration tương đương. Phân hệ DevOps Lab là ngoại lệ có chủ đích: phân hệ này cần k3s/Sysbox để cô lập môi trường thực hành, chứ không phải vì toàn bộ hệ thống đã đòi hỏi một nền tảng Kubernetes quy mô doanh nghiệp.

❗ KBM: Docker Compose Đủ Cho Production Nhỏ-Trung

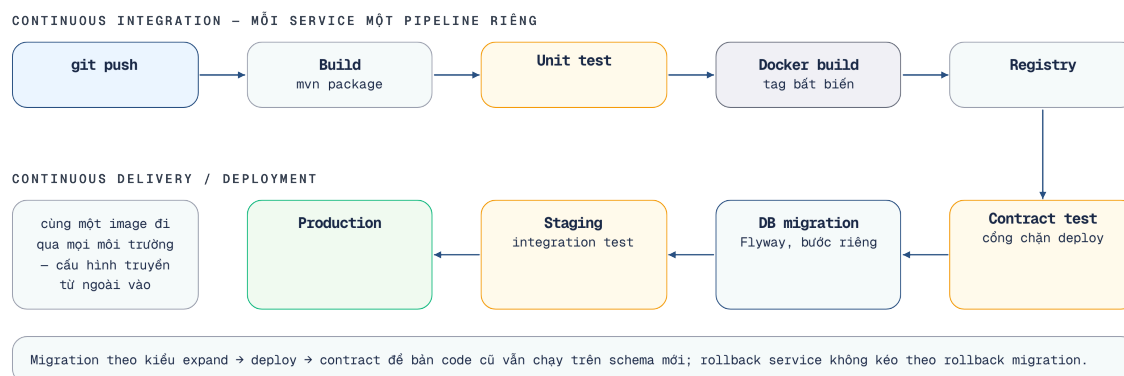
KBM LMS chính dùng Docker Compose thay vì Kubernetes — đây là lựa chọn **hợp lý cho giai đoạn hiện tại**. Docker Compose giảm đáng kể chi phí vận hành (không quản lý etcd, kube-apiserver, node pools) và đủ cho việc triển khai trên một máy chủ với tải vừa phải. Kubernetes chỉ cần thiết khi cần mở rộng trên nhiều máy chủ, có yêu cầu tự phục hồi, hay đội ngũ đủ lớn để vận hành cluster. Chọn Kubernetes trước khi thực sự cần là thiết kế quá mức cần thiết — không phải giải pháp khuyến nghị.

12.4. CI/CD Pipeline cho Microservices

Vấn đề: $N \text{ services} \times M \text{ stages} = \text{phức tạp}$ Trong kiến trúc monolith, pipeline CI/CD thường chỉ là một luồng duy nhất. Với microservices, mỗi dịch vụ sở hữu một vòng đời riêng, đòi hỏi pipeline tự động hóa phải xử lý được các ràng buộc phức tạp:

- Việc triển khai phiên bản mới của Service A **không được phép làm gián đoạn** Service B (API versioning — Ch.3)
- Database migration chạy **trước** service deploy
- Rollback service **không** được kéo theo rollback migration: schema phải tương thích ngược (expand → deploy → contract, xem phần Database Migration) để phiên bản code cũ vẫn chạy được trên schema đã mở rộng — down migration chỉ dùng khi đã chứng minh được tính khả đảo và có backup

12.4.1. Kiến trúc CI/CD cho Microservices



Hình 12.3: CI/CD Pipeline chuẩn cho Microservices

💡 Luồng CI/CD / Đề án tốt nghiệp

Toàn bộ CI/CD Pipeline có thể ví như quá trình nộp đề án:

Build – Đóng quyền đề án và dán mã số sinh viên bất biến (tương tự Image Tag).

Kiểm thử tự động – Phòng đào tạo tự động chạy hệ thống quét đạo văn Turnitin và kiểm tra định dạng văn bản.

Deploy Staging – Sinh viên bảo vệ thử nghiệm đề án trước Hội đồng Bộ môn.

Deploy Production – Sau khi duyệt, quyền đề án chính thức được đưa lên kệ Thư viện trường. Đặc biệt, tính **Bất biến (Immutable)** nghĩa là: khi phát hiện lỗi, sinh viên **không được chỉnh sửa trực tiếp trên bản đề án đã nộp lưu chiểu** (không SSH vào server sửa code). Mọi thay đổi phải đi từ bản nháp, in bản mới và làm lại toàn bộ quy trình nộp duyệt.

12.4.2. Mono-repo vs Poly-repo — Ảnh hưởng đến CI/CD

Câu hỏi đầu tiên khi thiết kế CI/CD là lưu trữ mã nguồn ở đâu. Giới công nghệ chia làm hai trường phái: **Mono-repo** (như cách Google, Meta làm) tích hợp tất cả các dịch vụ vào chung một kho lưu trữ, giúp chia sẻ mã nguồn và thực hiện các commit xuyên service (atomic changes) rất tiện lợi. Tuy nhiên, đánh đổi lại là một đường ống CI/CD đồ sộ, đòi hỏi logic phức tạp để chỉ biên dịch những dịch vụ có sự thay đổi về mã nguồn. Ngược lại, **Poly-repo** (được Netflix, Amazon ưa chuộng) phân tách mỗi dịch vụ thành một kho lưu trữ độc lập. Cách tiếp cận này giúp thiết lập N pipelines độc lập dễ dàng, nhưng khiến quá trình dùng chung mã nguồn (như các lớp DTO chia sẻ) phức tạp hơn nhiều và tốn kém nỗ lực vì phải xuất bản các thư viện trung gian nội bộ.

Mitra trong [3, Ch.6] khuyến nghị: đối với đội ngũ quy mô vừa và nhỏ, **mono-repo là lựa chọn đơn giản hơn** — giúp tránh chi phí quản lý (overhead) cho nhiều kho lưu trữ và luồng pipelines. Khi đội ngũ lớn (>20 lập trình viên), poly-repo mang lại mức độ tự chủ (autonomy) cao hơn cho các nhóm.

12.4.3. Tối ưu hóa Pipeline

Các giải pháp khuyến nghị trong việc thiết kế CI/CD Pipeline bao gồm:

Biên dịch một lần, triển khai mọi nơi (Build once, deploy everywhere) – Artifact (ví dụ Docker image) chỉ được biên dịch một lần duy nhất, sau đó được lần lượt đẩy (promote) qua các môi trường staging và production. Cách làm này dùng chung một artifact duy nhất, triệt tiêu rủi ro “thử nghiệm chạy tốt nhưng production lỗi” do mã nguồn bị biên dịch lại.

Cấu hình bên ngoài (externalized config) – Toàn bộ cấu hình đặc thù của môi trường (như URL cơ sở dữ liệu, API keys) phải được truyền vào ứng dụng thông qua biến môi trường. Container image phải luôn đảm bảo tính bất biến (immutability) bất kể nó đang được triển khai ở đâu.

Pipeline song song (parallel pipelines) – Mỗi service phải sở hữu khả năng đóng gói, kiểm thử và phân phối hoàn toàn độc lập. Việc nâng cấp Service A không được trở thành điểm nghẽn (bottleneck) cản trở quá trình phát hành của Service B.

Chạy database migration như một bước riêng – Công cụ quản lý cập nhật schema như Flyway hay Liquibase cần được thực thi như một bước độc lập trước khi tiến hành khởi chạy ứng dụng. Việc bóc tách sự thay đổi cấu trúc lưu trữ khỏi thay đổi logic mã nguồn sẽ làm cho quá trình phục hồi (rollback) trở nên rõ ràng và an toàn hơn đáng kể.

Cổng kiểm thử hợp đồng (contract test gate) – Trong một hệ sinh thái giao tiếp chéo phức tạp, pipeline chỉ được phép deploy khi toàn bộ contract tests (kiểm thử khế ước) đều đạt. Đây được xem là lá chắn cuối cùng giúp ngăn chặn các breaking changes gây lỗi trên môi trường production.

! Build Once, Deploy Everywhere

“Build artifact một lần duy nhất. Đẩy (promote) cùng artifact đó qua các môi trường (dev → staging → production). Nếu mỗi môi trường build một kiểu, nghĩa là đang kiểm thử một artifact khác với production.”

— Nguyên tắc CI/CD (Newman [4a, Ch.6], Mitra [3, Ch.10])

Đảm bảo tính bất biến của artifact qua các môi trường là cơ sở để xây dựng hệ thống tự động hóa đáng tin cậy. Nhờ đó, quy trình phát hành không còn phụ thuộc vào các thao tác thủ công, vốn là một hạn chế lớn khi số lượng dịch vụ gia tăng.

① Triển khai thủ công trở nên bất khả thi với microservices

Với N services độc lập, mỗi service có migration riêng, versioning riêng, và rollback plan riêng — việc triển khai thủ công rất chậm và **khó bảo trì khi mở rộng quy mô**. Mỗi thao tác thủ công đều tiềm ẩn rủi ro: sai thứ tự cập nhật lược đồ dữ liệu (migration), thiếu kịch bản khôi phục (rollback), hay lỗi do con người (human error). Tự động hóa CI/CD chuyển đổi việc triển khai thành một quy trình thường quy — và nghiên cứu DORA cho thấy việc triển khai thường xuyên hơn thực tế **giúp giảm thiểu tỷ lệ lỗi** (xem Ch.1, DORA Metrics).

12.4.4. Ví dụ Thực tế: CI/CD pipeline cho KBM

KBM thực tế vận hành trên **GitLab CI** (kèm GitLab Container Registry). Tuy nhiên, để cụ thể hóa các giải pháp khuyến nghị trên bằng một nền tảng phổ biến hơn với đa số độc giả, ví dụ dưới đây trình bày pipeline bằng **GitHub Actions** cho **kbm-academic** — các bước (build → test → scan → push → deploy) ánh xạ 1:1 sang **.gitlab-ci.yml** (stages/jobs). Mỗi service có workflow riêng; workflow chỉ kích hoạt khi có thay đổi trong thư mục của service đó (path filter) — tránh build lại toàn bộ repo khi chỉ sửa một service.

```

# .github/workflows/kbm-academic.yml
name: kbm-academic CI/CD

on:
  push:
    branches: [main]
    paths: [kbm-academic/**] # chỉ trigger khi service này thay đổi

env:
  REGISTRY: ghcr.io
  IMAGE: ${github.repository}/kbm-academic

jobs:
  build-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Set up JDK 21
        uses: actions/setup-java@v4
        with:
          java-version: '21'
          distribution: 'temurin'
          cache: 'maven'

      - name: Build and test
        run: mvn -pl kbm-academic -am verify

      - name: Log in to registry
        uses: docker/login-action@v3
        with:
          registry: ${env.REGISTRY}
          username: ${github.actor}
          password: ${secrets.GITHUB_TOKEN} # tự động từ GitHub Actions

      - name: Build image (local)
        uses: docker/build-push-action@v5
        with:
          context: kbm-academic
          load: true
          tags: ${env.REGISTRY}/${env.IMAGE}:${github.sha}

      - name: Scan vulnerability
        uses: aquasecurity/trivy-action@master
        with:
          image-ref: '${env.REGISTRY}/${env.IMAGE}:${github.sha}'
          format: 'table'
          exit-code: '1'
          severity: 'CRITICAL,HIGH'

      - name: Push image
        run: |
          # Push ĐÚNG image đã scan ở bước trước (không build lại) – giữ nguyên tắc
          "build once"
          docker tag ${env.REGISTRY}/${env.IMAGE}:${github.sha} \
            ${env.REGISTRY}/${env.IMAGE}:latest
          docker push ${env.REGISTRY}/${env.IMAGE}:${github.sha}
          docker push ${env.REGISTRY}/${env.IMAGE}:latest

```

```

deploy-staging:
  needs: build-test
  runs-on: ubuntu-latest
  environment: staging
  steps:
    - name: Run DB migration
      uses: appleboy/ssh-action@v1
      with:
        host: ${ secrets.STAGING_HOST }
        username: ${ secrets.STAGING_USER }
        key: ${ secrets.STAGING_SSH_KEY }
        script: |
          # BẢO MẬT: Inject qua file .env thay vì command line để tránh lộ secret
          trong `ps aux`
          set -Eeuo pipefail # dừng ngay khi có lệnh lỗi – exit
          code thất bại lan về pipeline
          trap 'rm -f .env.migration' EXIT # luôn dọn file secret, kể cả khi
          migration thất bại
          echo "SPRING_DATASOURCE_URL=${ secrets.STAGING_DB_URL }"
          > .env.migration
          docker run --rm --env-file .env.migration \
            ${ env.REGISTRY }/${ env.IMAGE }:${ env.github.sha } \
            --spring.flyway.enabled=true \
            --spring.main.web-application-type=none

    - name: Deploy service
      uses: appleboy/ssh-action@v1
      with:
        host: ${ secrets.STAGING_HOST }
        username: ${ secrets.STAGING_USER }
        key: ${ secrets.STAGING_SSH_KEY }
        script: |
          IMAGE_TAG=${ env.github.sha } \
          docker compose -f /app/docker-compose.yml \
          up -d --no-deps --wait kbm-academic # BẢO ĐẢM: --wait để chặn
          pipeline cho đến khi Health Check xanh

deploy-production:
  needs: deploy-staging
  runs-on: ubuntu-latest
  environment: production # yêu cầu manual approval trong GitHub Environments
  steps:
    - name: Deploy to production
      uses: appleboy/ssh-action@v1
      with:
        host: ${ secrets.PROD_HOST }
        username: ${ secrets.PROD_USER }
        key: ${ secrets.PROD_SSH_KEY }
        script: |
          IMAGE_TAG=${ env.github.sha } \
          docker compose -f /app/docker-compose.yml \
          up -d --no-deps kbm-academic

```

Listing 12.4: GitHub Actions pipeline cho `kbm-academic` — build → test → push → deploy

Một số điểm cần chú ý:

- `paths: [kbm-academic/**]` — sử dụng bộ lọc đường dẫn (path filter) đảm bảo kiến trúc mono-repo không tiến hành biên dịch lại toàn bộ hệ thống khi chỉ có thay đổi tại một dịch vụ cụ thể.
- DB migration (`deploy-staging: Run DB migration`) chạy trước khi deploy service, không phải sau. Thứ tự này là bắt buộc — xem mục tiếp theo.
- `environment: production` kết hợp GitHub Environments cho phép đặt người duyệt bắt buộc (required reviewers): mỗi lần deploy lên production cần con người phê duyệt.
- `github.sha` làm image tag thay vì `latest` — đảm bảo “build once, deploy everywhere”: cùng một artifact được promote từ staging lên production, không build lại.

12.4.5. Quản lý Secrets

Workflow trên dùng nhiều secrets (`STAGING_HOST` , `PROD_SSH_KEY` , `STAGING_DB_URL` ...). Nguyên tắc cơ bản:

 **Không gắn cứng thông tin xác thực**

Đặt thông tin xác thực (credentials) trực tiếp trong file `.yaml` là lỗi nghiêm trọng — file workflow được commit vào git history và có thể bị công khai. Dùng GitHub Secrets (Settings → Secrets and variables → Actions) để lưu trữ. `${{ secrets.GITHUB_TOKEN }}` là token tự động — không cần tạo thủ công.

Cụ thể, danh sách các secrets thiết yếu cho pipeline ví dụ ở trên được phân định rõ ràng về mặt phạm vi (Scope):

- `GITHUB_TOKEN` : Được sử dụng để xác thực tự động với kho chứa Registry. Token này được chính GitHub Actions tự động sinh ra và duy trì vòng đời ở phạm vi Repository mà không cần sự can thiệp thủ công.
- `STAGING_SSH_KEY` và `PROD_SSH_KEY` : Là các khóa mã hóa riêng tư (SSH private key) dùng để kết nối vào các máy chủ tương ứng. Chúng được giới hạn quyền truy cập ở phạm vi Environment (lần lượt cho môi trường staging và production).
- `STAGING_DB_URL` và `PROD_DB_URL` : Lưu trữ chuỗi kết nối (JDBC URL) đến các cụm cơ sở dữ liệu. Các thông số này cũng được cấu hình nghiêm ngặt theo từng phạm vi Environment tương ứng để loại trừ rủi ro truy cập chéo giữa các môi trường.

Lưu ý: secrets gắn với **Environment** (staging/production) thay vì ở mức Repository khi cần cô lập — staging secrets không thể dùng trong production job và ngược lại. Đây là cách GitHub Environments bảo đảm cô lập giữa các môi trường triển khai.

12.4.6. Database Migration trong CI/CD Pipeline

Trong microservices, schema change và code change là hai artifact độc lập — nhưng phải được phối hợp theo thứ tự chính xác. Sai thứ tự dẫn đến downtime hoặc hỏng dữ liệu (data corruption).

12.4.6.1. Vấn đề cốt lõi: code cũ đọc schema mới

Trong quá trình cập nhật cuốn chiếu (rolling update), luôn tồn tại một khoảng thời gian mà **cả phiên bản cũ và mới** của service cùng hoạt động đồng thời. Nếu migration tạo schema không tương thích ngược:

- Dịch vụ cũ v1 (đang xử lý request) đọc cột vừa bị đổi tên sẽ dẫn đến lỗi.
- Dịch vụ mới v2 (vừa được triển khai) đọc cột mới sẽ hoạt động bình thường.
- Kết quả: một phần request thành công, một phần lỗi — khó debug, khó rollback.

12.4.6.2. Expand/Contract Pattern — Migration tương thích ngược

Giải pháp chuẩn mực cho bài toán này là chia nhỏ schema migration thành ba bước theo chuẩn Expand/Contract Pattern. Bắt đầu với bước **Expand (Mở rộng)**, chúng ta chỉ thêm các bảng/cột mới mà tuyệt đối không động chạm đến cột cũ, đảm bảo cả phiên bản dịch vụ v1 (cũ) và v2 (mới) đều hoạt động ổn định. Kế đến là bước **Chuyển dữ liệu** (migrate data), sử dụng các background job để chuyển dần dữ liệu (backfill) từ cấu trúc cũ sang cấu trúc mới mà không làm gián đoạn hệ thống. Cuối cùng, khi 100% traffic đã đổ về service v2 và v1 đã được gỡ bỏ, chúng ta tiến hành bước **Contract (Thu hẹp)** để dọn dẹp các cột/bảng thừa thãi.

Ví dụ minh họa trong `kbm-academic`: đổi tên cột `score` thành `score_10` (thang 10):

```
-- Giai đoạn 1 (Expand): thêm cột mới (score_10), GIỮ cột cũ (score)
ALTER TABLE grades ADD COLUMN score_10 DECIMAL(4,2);
-- CẢNH BÁO: lệnh backfill dưới đây KHÔNG chạy đồng bộ trong migration CI/CD
-- (UPDATE toàn bảng lớn sẽ lock bảng, treo pipeline, phá zero-downtime).
-- Chạy bằng batch job nền sau khi migrate schema xong, cập nhật theo lô nhỏ:
UPDATE grades SET score_10 = score WHERE score_10 IS NULL; -- backfill (batch job nền)

-- Deploy service v2 (đọc score_10, vẫn ghi đồng bộ cả hai)
-- Đợi đến khi 100% traffic đã sang v2 và v1 đã dừng

-- Giai đoạn 2 (Contract): xóa cột cũ
ALTER TABLE grades DROP COLUMN score; -- chỉ thực thi sau khi v1 (code cũ) ngừng hoạt động hoàn toàn
```

12.4.6.3. Thứ Tự Thực Hiện trong Pipeline

⚠ Migration Trước Khi Deploy

Thứ tự bắt buộc: **migration → deploy new service → (sau khi stable) retire old schema**. Nếu triển khai dịch vụ trước khi cập nhật cơ sở dữ liệu, dịch vụ mới sẽ gặp cấu trúc dữ liệu cũ, dẫn đến lỗi khởi động (startup failure). Flyway và Liquibase kiểm tra checksum và version trước khi service nhận request — đây là lý do bước migration trong 12.4 được tách thành một step riêng, không chạy trong `application startup`.

Trong 12.4, migration được chạy qua `docker run --rm` với flag `--spring.main.web-application-type=none` — service khởi động, chạy Flyway, rồi thoát ngay mà không bind port. Chỉ sau khi bước này thành công, pipeline mới deploy service thật sự. Đây là pattern phổ biến với Spring Boot + Flyway. Ngoài ra, cần lưu ý có cơ chế xử lý lỗi: phải có kịch bản rollback script hoặc dừng pipeline yêu cầu can thiệp thủ công (manual intervention) nếu script SQL chạy lỗi, nhằm tránh treo pipeline hoặc làm hỏng database.

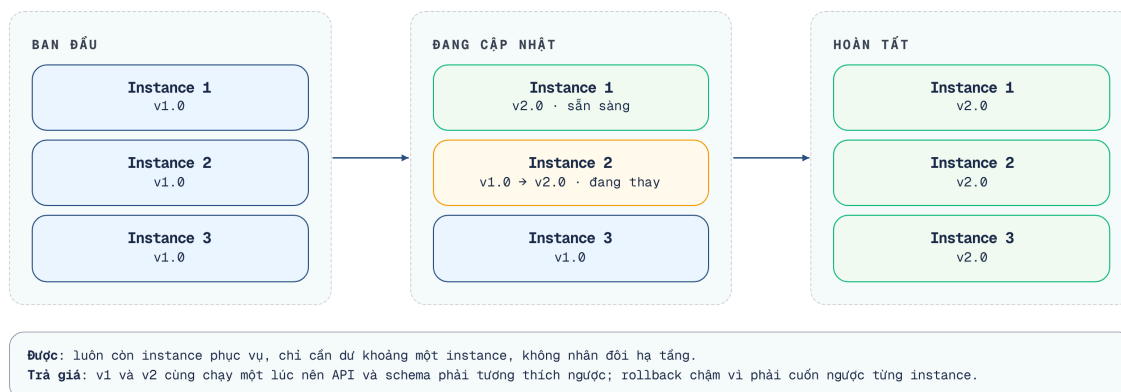
12.5. Các chiến lược triển khai

Vấn đề: triển khai phiên bản mới mà không gây downtime Khi triển khai phiên bản mới của một service, làm thế nào để đảm bảo người dùng không bị gián đoạn? Nếu phiên bản mới tồn tại lỗi, làm thế nào để khôi phục hệ thống một cách nhanh chóng?

Bốn chiến lược được trình bày dưới đây: ba cách phát hành (Rolling, Blue/Green, Canary) và một kỹ thuật kiểm chứng trước khi phát hành (Shadow).

12.5.1. Rolling Update

Thay thế lần lượt **từng thực thể (instance)** — chuyển đổi dần dần từ phiên bản cũ sang phiên bản mới mà không yêu cầu tăng gấp đôi tài nguyên hạ tầng.



Hình 12.4: Quá trình diễn ra Rolling Update

12.5.2. Blue/Green Deployment

Chạy **hai bản hoàn chỉnh** song song — “Blue” (hiện tại) và “Green” (mới). Router chuyển traffic một lần. Rollback = chuyển router ngược lại. Trong Kubernetes, mô hình này thường được thực hiện bằng cách thay đổi **selector** của **Service** để trỏ từ phiên bản Blue sang Green. Quá trình này diễn ra tức thời mà không cần cấu hình tỷ lệ traffic.

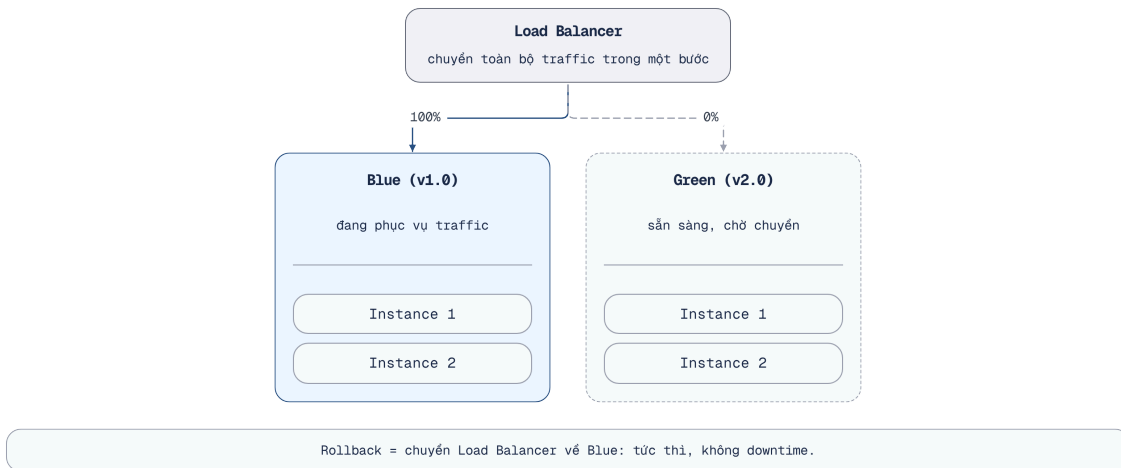
```
# Triển khai phiên bản Green (đứng chờ, chưa có traffic)
kubectl apply -f deployment-green.yaml

# Khi test trên Green thành công, tiến hành switch traffic lập tức bằng cách patch
Service
kubectl patch service kbm-academic-svc -p '{"spec":{"selector":
{"version":"green"}}}'

# Rollback ngay lập tức nếu có lỗi:
kubectl patch service kbm-academic-svc -p '{"spec":{"selector":{"version":"blue"}}}'
```

Listing 12.5: Chuyển traffic lập tức bằng cách patch Service trong Blue/Green

Việc thay đổi luồng định tuyến (routing) ở cấp độ mạng giúp hệ thống ngay lập tức phục vụ phiên bản mới mà không tốn thời gian khởi động hoặc làm gián đoạn các kết nối hiện tại.



Hình 12.5: Kiến trúc Blue/Green Deployment

12.5.3. Canary Release

Triển khai phiên bản mới cho **một phần nhỏ lưu lượng truy cập**, sau đó giám sát các chỉ số đo lường (metrics) và tiến hành mở rộng dần nếu hệ thống hoạt động ổn định. Trong môi trường Kubernetes sử dụng Nginx Ingress, chiến lược Canary có thể được hiện thực hóa bằng cách cấu hình annotation điều hướng một tỷ lệ phần trăm traffic (ví dụ 10%) sang phiên bản mới trong khi 90% vẫn vào phiên bản ổn định.

```
# canary-ingress.yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: kbm-academic-canary-ingress
  annotations:
    nginx.ingress.kubernetes.io/canary: "true"
    nginx.ingress.kubernetes.io/canary-weight: "10" # Định tuyến 10% traffic vào
    Canary
spec:
  rules:
  - host: api.example.edu
    http:
      paths:
      - path: /assignment
        pathType: Prefix
        backend:
          service:
            name: kbm-academic-canary-svc
            port:
              number: 80
```

Listing 12.6: Cấu hình Canary Ingress phân luồng 10% traffic

12.5.4. Shadow Deployment

Gửi **một bản sao** của traffic thật sang phiên bản mới, trong khi người dùng vẫn chỉ nhận kết quả từ phiên bản cũ. Phiên bản mới xử lý request thật nhưng phản hồi của nó bị loại bỏ — nên lỗi ở đó không chạm tới ai. Kỹ thuật này còn được gọi là *traffic mirroring* hoặc *dark launching*.

Điểm mạnh là nó trả lời được câu hỏi mà staging không trả lời nổi: phiên bản mới chịu tải *thật* có ổn không. Đổi lại, chi phí cao nhất trong bốn chiến lược — phải chạy song song đủ tài nguyên cho toàn bộ traffic, và mọi **tác dụng phụ** (side effect) phải được cô lập. Nếu bản shadow vẫn ghi vào cùng database hay vẫn gửi email, nó sẽ nhân đôi hậu quả thay vì mô phỏng chúng.

Với KBM, ứng dụng hợp lý nhất là bộ chấm bài: cho **judge** phiên bản mới chấm song song cùng bộ bài nộp, so kết quả với bản cũ, nhưng chỉ bản cũ mới được ghi điểm cho sinh viên.

12.5.5. So sánh

So sánh bốn chiến thuật triển khai: **Rolling Update** giúp đạt zero-downtime với chi phí tài nguyên cực thấp (chỉ cần dư một instance) nhưng có nhược điểm là quá trình rollback chậm. **Blue/Green** cung cấp khả năng khôi phục (rollback) tức thời chỉ thông qua việc điều hướng lại router, phù hợp cho các đợt phát hành rủi ro cao nhưng yêu cầu chi phí vận hành gấp đôi (2x infrastructure). **Canary Release** cung cấp cơ chế rollback tức thì và chi phí tài nguyên vừa phải, đồng thời cho phép thử nghiệm trên một nhóm nhỏ người dùng trước khi mở rộng (cơ chế chia lưu lượng giống A/B testing, nhưng mục đích là giảm rủi ro).

Shadow Deployment không thay người dùng sang bản mới lần nào, nên nó không phải là một cách phát hành — nó là một cách *kiểm chứng trước khi* phát hành, và thường đứng trước một trong ba chiến lược kia.

Newman trong [4a, Ch.6] khuyến nghị: chọn chiến lược dựa trên **mức độ rủi ro** của thay đổi. Các bản vá lỗi nhỏ gọn nên áp dụng rolling update. Các thay đổi lớn có tầm ảnh hưởng diện rộng cần sử dụng blue/green. Những tính năng mới mang tính thử nghiệm nên được triển khai theo mô hình canary. Khi rủi ro nằm ở *hiệu năng dưới tải thật* chứ không ở tính đúng đắn của logic, shadow deployment là cách đo trước mà không bắt người dùng gánh rủi ro.

12.6. Infrastructure as Code (IaC)

Việc cấu hình máy chủ thủ công qua SSH dễ gây ra tình trạng sai lệch cấu hình (configuration drift) giữa các môi trường, khiến chúng không thể tái tạo lại (reproducible). Infrastructure as Code (IaC) là giải pháp tiêu chuẩn cho vấn đề này.

12.6.1. Vấn đề: “Ai cấu hình server? Config ở đâu?”

Khi hệ thống microservices chạy trên nhiều servers/containers, cấu hình thủ công dẫn đến sai lệch cấu hình (configuration drift) nghiêm trọng — một máy chủ có thể bị thay đổi mà không được đồng bộ với máy chủ khác. Tình trạng này khiến môi trường không thể được tái tạo chính xác vì không ai nhớ rõ các bước thiết lập ban đầu. Hậu quả tồi tệ nhất là khi xảy ra lỗi, hệ thống hoàn toàn mất đi khả năng quay lui vì không lưu giữ được trạng thái cấu hình ổn định trước đó.

12.6.2. IaC — Hạ tầng như Code

Infrastructure as Code (IaC) nghĩa là mọi hạ tầng — servers, networks, databases, message brokers — được **định nghĩa trong code** (thường là declarative YAML/HCL), được quản lý phiên bản (version controlled), và deploy tự động.

Bên cạnh CI/CD pipeline truyền thống, cộng đồng cloud-native đang mở rộng sang ba hướng bổ sung:

GitOps (ArgoCD, Flux) – Lấy Git repository làm nguồn sự thật duy nhất (single source of truth) và tự động đồng bộ trạng thái hạ tầng, thay cho việc áp cấu hình thủ công.

Multi-cluster (Rancher, Karmada) – Quản lý nhiều cụm ở nhiều vùng cho các tổ chức quy mô toàn cầu, đổi lại là bài toán kết nối mạng liên cụm.

Supply chain security – Các khung kiểm định như SLSA và công cụ ký artifact như Sigstore, đi kèm SBOM (Software Bill of Materials) — bản kê thành phần phần mềm — đang dần trở thành yêu cầu chuẩn.

Các xu hướng này đều *bổ sung* — không thay thế — pipeline CI/CD đã trình bày trong chương này. Mitra trong [3, Ch.6-7] mô tả IaC là nền tảng cho việc triển khai microservices thông qua một hệ sinh thái công cụ đa dạng, phục vụ cho từng tầng khác nhau của kiến trúc:

Tầng Provisioning (Cung cấp tài nguyên) – Bao gồm các công cụ tiêu biểu như Terraform, Pulumi hay CloudFormation. Chúng đảm nhận nhiệm vụ khởi tạo hạ tầng gốc rễ như việc khởi tạo hệ thống máy ảo (VMs), cấu hình mạng nội bộ (VPCs) hay cung cấp cơ sở dữ liệu dạng managed.

Tầng Container Orchestration (Điều phối container) – Sử dụng các mô tả dạng Kubernetes manifests hay Helm charts để định nghĩa cách thức điều phối ứng dụng. Tại tầng này, các quy tắc chi tiết về việc mở rộng (scaling rules) và kiểm soát định tuyến được phác họa đầy đủ.

Tầng Configuration Management (Quản lý cấu hình) – Các công cụ như Ansible, Chef hay Puppet hỗ trợ đắc lực trong việc cấu hình hệ điều hành cơ sở và cài đặt các thư viện hệ thống mức host.

Tầng Service Deployment (Triển khai dịch vụ) – Những nền tảng như Docker Compose, Kubernetes hay ArgoCD trực tiếp quản lý vòng đời của các dịch vụ, đảm bảo toàn bộ ngăn xếp microservices luôn được duy trì ở trạng thái khai báo chuẩn mực.

Thay vì quản lý các file YAML thô cứng, các dự án Kubernetes thực tế thường sử dụng **Helm** làm trình quản lý gói (package manager) để biến manifest thành các template có thể tái sử dụng dễ dàng giữa các môi trường:

```
# Cài đặt hoặc nâng cấp kbm-academic service với custom values cho production
helm upgrade --install kbm-academic ./kbm-service \
  --namespace kbm-prod \
  --set image.tag=v2.0.1 \
  --set replicaCount=5 \
  --set resources.limits.cpu=1000m
```

Listing 12.7: Quản lý Kubernetes IaC với Helm

12.6.3. Docker Compose như một dạng IaC

Docker Compose — dù đơn giản — đã là một dạng IaC: hạ tầng (databases, brokers) và services đều định nghĩa trong file YAML, được quản lý phiên bản, tái tạo được.

Việc áp dụng IaC thường trải qua ba cấp độ trưởng thành rõ rệt. Bắt đầu ở **Cấp 1**, hệ thống đơn giản dựa vào *Docker Compose* để đáp ứng các nhu cầu trên một máy chủ đơn lẻ (single-host), môi trường phát triển hay thực tế ở quy mô nhỏ. Khi khối lượng xử lý gia tăng nhanh chóng đòi hỏi khả năng tự động mở rộng và tự phục hồi (auto-scaling, self-healing) và dàn trải trên nhiều máy chủ, hệ thống tiến lên **Cấp 2** với *Kubernetes* và *Helm*. Ở **Cấp 3**, quy trình đạt đến mô hình *Full GitOps* bằng cách kết hợp *Terraform*, *Kubernetes* và *ArgoCD*, nơi cả cấu trúc hạ tầng lẫn phần mềm dịch vụ đều được khai báo 100% dưới dạng code và tự động đồng bộ hóa.

Với KBM LMS chính — hệ thống giáo dục quy mô trung bình — **Cấp 1 (Docker Compose)** hiện vẫn phù hợp. DevOps Lab dùng một cụm k3s nhỏ cho bài toán cô lập môi trường lab riêng. Khi toàn hệ thống cần mở rộng trên nhiều máy chủ, tự phục hồi và rolling deployment đồng đều, mới chuyển dần lên Cấp 2 (Kubernetes).

Infrastructure as Code

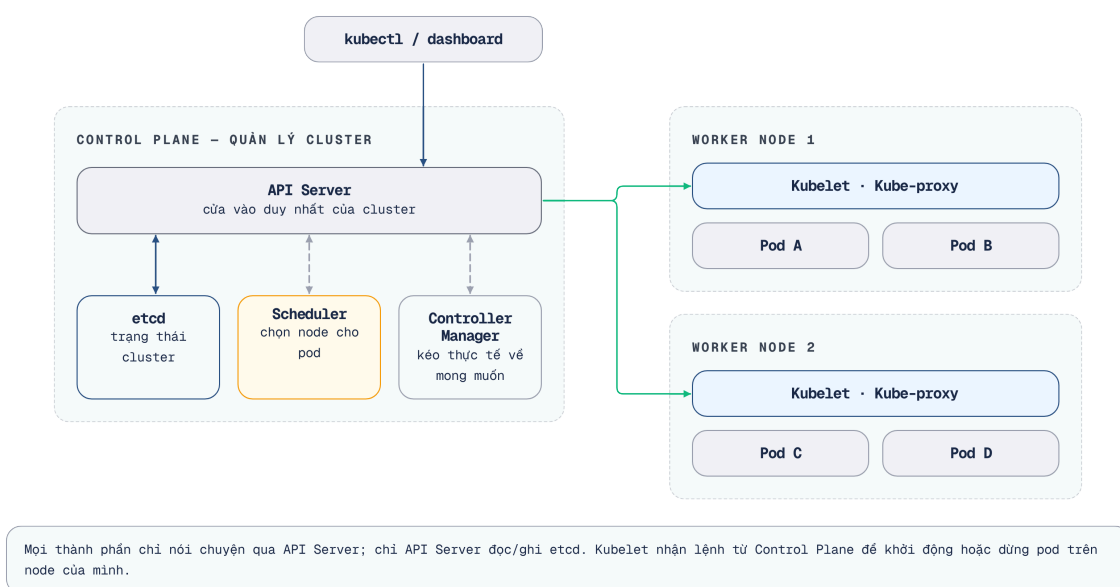
“If you can’t reproduce your infrastructure from scratch using code, you don’t truly own your infrastructure — you’re just renting it from whoever set it up last.”

— Mitra [3, Ch.6], nguyên tắc IaC

12.6.4. Kubernetes — Container Orchestration cho Production

Docker Compose phù hợp cho môi trường phát triển và hệ thống nhỏ, ít yêu cầu orchestration. Khi cần **multi-node deployment, auto-scaling, self-healing**, Kubernetes (K8s) trở thành nền tảng tiêu chuẩn. Newman trong [4a, Ch.6] nhận xét: “Kubernetes has become the de facto platform for running microservices at scale.”

Kiến trúc Kubernetes phân tán. Kiến trúc của hệ thống này được chia thành hai phần chính:



Hình 12.6: Kiến trúc Kubernetes — Control Plane và Worker Nodes

Trong đó, **Control Plane** có chức năng quản lý toàn bộ cluster: API Server nhận requests (từ **kubect1** hoặc dashboard), etcd lưu toàn bộ trạng thái cluster, Scheduler quyết định pod chạy trên node nào, Controller Manager đảm bảo trạng thái thực tế = trạng thái mong muốn. Còn lại, các **Worker Nodes** chịu trách nhiệm chạy workloads: Kubelet trên mỗi node nhận lệnh từ Control Plane để khởi động hoặc dừng các pods tương ứng.

12.6.4.1. Các khái niệm cốt lõi

Pod – Là đơn vị tính toán nhỏ nhất được quản lý trong Kubernetes. Một Pod có thể chứa một hoặc nhiều containers cùng chia sẻ chung một mạng lưới nội bộ (network namespace) và bộ nhớ lưu trữ (storage). Về mặt ánh xạ, nó có thể ví như một khối **service** độc lập trong cấu hình Compose.

Deployment – Đối tượng cao cấp quản lý vòng đời của Pod. Nó giúp duy trì chính xác số lượng bản sao (replicas) mong muốn của ứng dụng và đảm trách quá trình cập nhật diễn ra không gián đoạn (rolling updates). Tính năng orchestration tĩnh này hoàn toàn vắng bóng trong Docker Compose.

Service – Tạo ra một network endpoint ổn định nhằm hỗ trợ load balancing luồng truy cập nội bộ, đồng thời giải quyết bài toán địa chỉ IP của các Pods bị thay đổi liên tục khi tái tạo.

Ingress – Là nút giao thông thông minh, chịu trách nhiệm định tuyến các luồng truy cập HTTP từ thế giới bên ngoài vào trong cụm dựa trên các bộ quy tắc tên miền (domain-based routing). Nó có chức năng tương đương với việc cấu hình thủ công một Nginx reverse proxy.

ConfigMap/Secret – Bóc tách lớp cấu hình ra khỏi vòng đời của mã nguồn ứng dụng. Secret API tách dữ liệu nhạy cảm khỏi manifest và kiểm soát truy cập bằng RBAC, nhưng giá trị Secret mặc định chỉ được Base64-encode — muốn mã hóa thật sự trong etcd phải bật encryption at rest hoặc dùng external secret manager. Ở phía Compose, KBM hiện inject một phần secret qua file `.env` thuần văn bản (phần còn lại vẫn nằm trong `application.yml`, Ch.9), dù bản thân Compose cũng có cơ chế `secrets` mount vào `/run/secrets`.

HPA (Horizontal Pod Autoscaler) – Thành phần hỗ trợ tự động thu phóng số lượng Pods linh hoạt dựa trên sự biến động của tải thực tế (như CPU, memory hoặc các custom metrics). HPA biến Kubernetes thành một hệ thống thực sự phản ứng nhạy bén với người dùng.

Dưới đây là file YAML cấu hình hoàn chỉnh cho một Service (bao gồm Deployment, Service, Ingress và HPA) áp dụng các giải pháp khuyến nghị như **Liveness Probe** và **Resource Limits**:

```
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: kbm-academic
spec:
  # Không khai báo replicas khi đã có HPA bên dưới: mỗi lần apply/GitOps sync
  # sẽ ép số Pod về giá trị cứng này rồi HPA lại kéo về, gây flapping.
  # replicas do HPA sở hữu.
  selector:
    matchLabels:
      app: kbm-academic
  template:
    metadata:
      labels:
        app: kbm-academic
    spec:
      containers:
        - name: kbm-academic
          image: ghcr.io/kbm/kbm-academic:v1.2.0
          ports:
            - containerPort: 8080
          resources:
            requests:
              cpu: "200m"
              memory: "512Mi"
            limits:
              cpu: "500m"
              memory: "1Gi"
          livenessProbe:
            httpGet:
              path: /actuator/health/liveness
              port: 8080
            initialDelaySeconds: 30
      # Không có readinessProbe thì kubelet đánh dấu Pod Ready ngay khi container
      # start; Service route traffic vào JVM chưa khởi tạo xong → 502 ở mỗi lần
```

```

    # rolling update. Đây chính là cơ chế tạo ra zero-downtime (xem §11.6).
    readinessProbe:
      httpGet:
        path: /actuator/health/readiness
        port: 8080
        periodSeconds: 5
        failureThreshold: 3
---
apiVersion: v1
kind: Service
metadata:
  name: kbm-academic-svc
spec:
  selector:
    app: kbm-academic
  ports:
    - protocol: TCP
      port: 80
      targetPort: 8080
---
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: kbm-academic-ingress
  annotations:
    nginx.ingress.kubernetes.io/use-regex: "true"
    # /$2 giữ lại phần URI sau prefix – rewrite-target: / sẽ cắt cụt mọi path con
    nginx.ingress.kubernetes.io/rewrite-target: /$2
spec:
  rules:
    - host: api.example.edu
      http:
        paths:
          - path: /assignment(/|$)(.*)
            pathType: ImplementationSpecific
            backend:
              service:
                name: kbm-academic-svc
                port:
                  number: 80
---
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: kbm-academic-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: kbm-academic
  minReplicas: 2
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:

```

```
type: Utilization
averageUtilization: 70
```

Listing 12.8: Cấu hình Kubernetes chuẩn mực (Production-grade) cho KBM Service

Docker Compose vs Kubernetes — So sánh chi tiết

Năng lực	Docker Compose	Kubernetes
Multi-host	× Chỉ một máy chủ	✓ Cluster nhiều nodes
Auto-scaling	× Thủ công <code>--scale</code>	✓ HPA dựa trên metrics
Self-healing	! <code>restart: always</code> (cơ bản)	✓ Liveness/readiness probes, tự thay thế
Rolling updates	× Gián đoạn khi khởi động lại	✓ Zero-downtime rolling update
Service discovery	✓ DNS theo tên service	✓ DNS + load balancing
Secret management	! Có <code>secrets</code> mount <code>/run/secrets</code> ; KBM hiện dùng <code>.env</code> plaintext và cả <code>application.yml</code>	✓ Secret API + RBAC (base64; bật encryption-at-rest để mã hóa thật)
Resource limits	! Cơ bản (deploy.resources)	✓ Requests + limits theo từng container
Độ phức tạp thiết lập	Thấp (1 YAML file)	Cao (nhiều YAML, thiết lập cluster)
Thời gian học	1-2 ngày	2-4 tuần
Quy mô đội ngũ cần thiết	1-3 người	3-5+ người (hoặc managed K8s)

Bảng 12.1: So sánh chi tiết Docker Compose và Kubernetes

12.6.4.2. Kịch bản chuyển đổi của KBM: Compose → Kubernetes

Nếu KBM cần mở rộng hơn (nhiều đơn vị đào tạo, nhiều sinh viên đồng thời):

```
Giai đoạn 1 (Hiện tại): Docker Compose cho LMS chính
├── Phù hợp hạ tầng nhỏ và workload vừa phải
├── Deploy: docker-compose up -d
└── Chi phí vận hành thấp, thao tác đơn giản

Giai đoạn 2 (Mở rộng): Managed Kubernetes (GKE/EKS/AKS)
├── Nhiều đơn vị đào tạo hoặc contest traffic lớn
├── Judge Service: HPA scale 1→10 pods khi contest
├── Core Service: 2-3 replicas cho high availability
├── PostgreSQL: managed service (Cloud SQL/RDS)
└── Chi phí và độ phức tạp tăng đáng kể
```

Listing 12.9: Kịch bản mở rộng KBM từ Compose lên Kubernetes

Không chuyển lên K8s vì “Netflix dùng”. Chuyển khi có **ít nhất 2 tiêu chí**: (1) cần multi-node deployment hoặc workload isolation tốt hơn, (2) cần auto-scaling (tải biến động: kỳ thi → bình thường), (3) cần zero-downtime deployment (SLA yêu cầu), (4) team đủ năng lực vận hành K8s. Managed Kubernetes (GKE, EKS, AKS) giảm đáng kể độ phức tạp — không cần tự dựng/bảo trì control plane.

12.6.5. Triển khai Serverless — Khi nào phù hợp?

Scale-to-Zero trong Kubernetes

Bên cạnh các dịch vụ Serverless thuần túy từ Cloud Provider, trên môi trường Kubernetes, xu hướng hiện nay là dùng **KEDA** (Kubernetes Event-driven Autoscaling) hoặc **Knative** để mang lại trải nghiệm tương tự Serverless: cho phép scale pods xuống 0 khi không có tải (Scale-to-Zero), giúp tối ưu chi phí cực lớn cho các dịch vụ chạy ngầm.

Richardson trong [2a, Ch.12] liệt kê **Serverless** (AWS Lambda, Google Cloud Functions, Azure Functions) là một mô hình triển khai cho microservices. Thay vì quản lý containers, chúng ta deploy *functions* — nhà cung cấp cloud quản lý hạ tầng, tự động mở rộng và tính phí theo lượt gọi.

Khía cạnh	Container (Docker/K8s)	Serverless
Quản lý server	Tự quản lý (hoặc managed K8s)	Cloud provider quản lý hoàn toàn
Mở rộng	Cấu hình quy tắc auto-scaling	Tự động mở rộng ($0 \rightarrow N$)
Mô hình chi phí	Trả theo máy chủ/giờ (dù có chạy hay không)	Trả theo lượt gọi (không dùng = không trả)
Cold start	Không (container luôn chạy)	Có (100ms-3s khởi động lần đầu)
Stateful	Có thể (volumes, sessions)	Chỉ phi trạng thái (stateless)
Giới hạn thời gian chạy	Không giới hạn	Thường tối đa 15 phút mỗi lượt gọi
Phụ thuộc nhà cung cấp (vendor lock-in)	Thấp (Docker dễ di chuyển)	Cao (API riêng mỗi cloud)

Bảng 12.2: Container (Docker/K8s) vs Serverless

Khi nào serverless phù hợp cho microservices?

Khối lượng công việc mang tính đột biến và hướng sự kiện (Event-driven, bursty workloads)

– xử lý tải lên tệp, thay đổi kích thước ảnh, gửi thông báo — không yêu cầu máy chủ duy trì hoạt động liên tục

Hàm kết nối (glue functions) – kết nối services, biến đổi dữ liệu, kích hoạt workflow

Chế bản mẫu (Prototype/MVP) – triển khai nhanh chóng mà không đòi hỏi thiết lập hạ tầng phức tạp

Khi nào KHÔNG phù hợp?

Các luồng xử lý thiết yếu yêu cầu độ trễ thấp – thời gian khởi động nguội không phù hợp cho các giao tiếp API đồng bộ (luồng nộp bài của LMS)

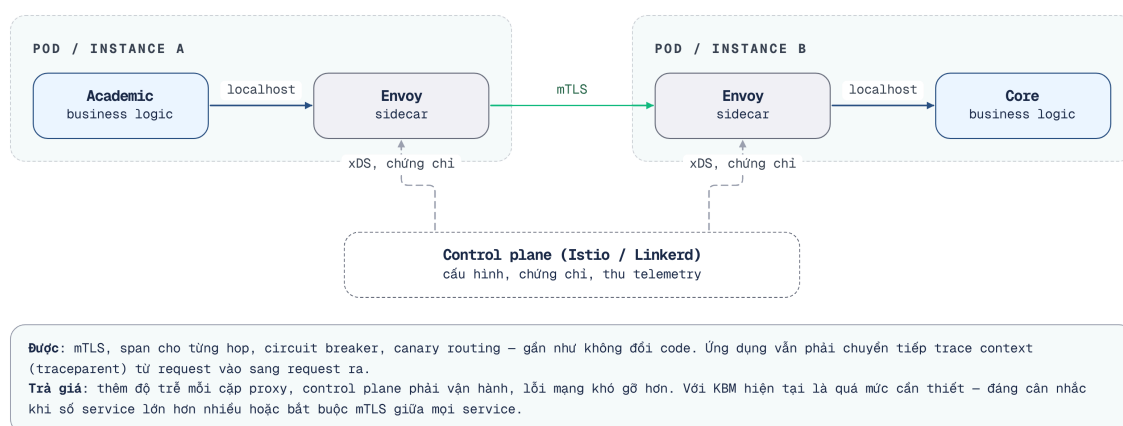
Tiến trình dài hoặc giữ tài nguyên cố định – Judge giữ kết nối tới các sandbox DBMS và mỗi bài chạy 5–30 giây, cộng hàng đợi dồn trong kỳ thi — không hợp mô hình hàm ngắn hạn tính tiền theo lượt gọi

Các dịch vụ lưu trữ trạng thái (Stateful services) – yêu cầu duy trì kết nối cơ sở dữ liệu hoặc bộ nhớ đệm, trong khi kiến trúc serverless buộc phải thiết lập lại kết nối sau mỗi lần kích hoạt

Trong LMS, phần **gửi email/push** khi có sự kiện là ứng viên tốt cho serverless: hướng sự kiện (nhận event từ Kafka), đột biến theo kỳ thi, không giữ trạng thái. Kênh WebSocket của Notification Service thì không — nó giữ kết nối lâu với trình duyệt. Core, Judge, Gateway phù hợp với containers hơn.

12.6.6. Sidecar Pattern và Service Mesh

Khi hệ thống microservices lớn (20+ services), mỗi service cần tự triển khai cùng các cross-cutting concerns: mTLS, logging, tracing, circuit breaker, rate limiting. **Sidecar pattern** giải quyết bằng cách đặt một **tiền trình proxy bên cạnh mỗi instance của service** — proxy xử lý các mối quan tâm hạ tầng, service chỉ tập trung vào business logic.



Hình 12.7: Sidecar Pattern — Proxy xử lý các mối quan tâm hạ tầng (mTLS, tracing)

Service Mesh (Istio, Linkerd) = sidecar proxies trên mọi service + control plane quản lý tập trung. Tự động cung cấp: mTLS giữa services, distributed tracing, traffic management (canary routing), circuit breaking — **gần như không cần thay đổi code** (ngoại lệ quan trọng của tracing được nói rõ bên dưới). Xu hướng gần đây là **sidecarless / ambient mesh** (Istio Ambient, Cilium Service Mesh dựa trên eBPF): mTLS và định tuyến L4 được xử lý ở tầng node/kernel thay vì chạy một proxy riêng cho mỗi pod, nhờ đó cắt bớt chi phí phụ trội (overhead) trên mỗi hop và chi phí vận hành sidecar — đáng cân nhắc nếu team đánh giá service mesh cho hệ thống mới.

Việc áp dụng Service Mesh mang đến một sự thay đổi rõ rệt trong việc gỡ rối các bài toán hạ tầng mạng so với cách làm phân mảnh truyền thống:

Bảo mật luồng giao tiếp (mTLS) – Thay vì buộc các lập trình viên phải tự triển khai logic mã hóa và quản lý chứng chỉ, các sidecar proxy của Service Mesh tự động thực hiện handshake và thiết lập kết nối mTLS giữa các microservices một cách trong suốt với ứng dụng.

Truy vết phân tán (Tracing) – Sidecar tự sinh span cho từng hop mạng và export về backend, giảm đáng kể phần hạ tầng phải tự dựng. Tuy nhiên, ứng dụng vẫn **bắt buộc** phải chuyển tiếp trace context (**traceparent** hoặc B3 headers) từ request đi vào sang request gửi ra. Proxy không thể tự suy ra outbound request nào bắt nguồn từ inbound request nào; thiếu bước này, các span sẽ không ghép được thành một trace hoàn chỉnh.

Cơ chế tự bảo vệ (Circuit Breaker) – Thay vì phải viết các cấu hình ngắt mạch qua Resilience4j lồng ghép vào business logic, mọi quy tắc đều được khai báo tập trung ở lớp Envoy proxy, qua đó loại bỏ logic xử lý lỗi kết nối khỏi ứng dụng.

Định tuyến nâng cao (Canary routing) – Quá trình phân luồng traffic được thực hiện thông qua các file cấu hình declarative, thay cho thao tác thủ công trên Load Balancer.

Đánh đổi về hiệu năng (Overhead) – Mặc dù giảm thiểu phức tạp trong mã nguồn, Service Mesh làm tăng độ trễ mạng (benchmark chính thức của Istio ghi nhận khoảng 2–3 ms ở P90 cho một cặp proxy; con số thực tế phụ thuộc cấu hình telemetry và tải, nên phải đo trên chính hệ thống của mình) do request phải đi qua các lớp proxy.

Với KBM hiện tại, việc áp dụng Service Mesh vẫn mang tính thiết kế quá mức (over-engineering) cho hệ thống LMS chính. Mặc dù DevOps Lab hướng hệ thống theo môi trường đa ngôn ngữ (polyglot Java+Go), nhu cầu thiết yếu trước mắt vẫn là hoàn thiện CI/CD, tính khả quan sát (observability) và quản lý cấu hình bảo mật. Service mesh phù hợp khi: số lượng service lớn hơn nhiều, multi-host deployment trở thành mặc định, hoặc yêu cầu bảo mật cao đến mức mTLS bắt buộc giữa mọi service.

⚠ Thời gian học và độ phức tạp khi xử lý sự cố

Dù Service Mesh giải quyết nhiều vấn đề cross-cutting, nhưng đổi lại là độ phức tạp lớn. Vận hành Control Plane (như Istio) và debug lỗi mạng ở tầng proxy/eBPF gây khó khăn lớn cho việc bảo trì nếu team không có chuyên gia mạng. Việc đưa Service Mesh vào khi chưa thực sự cần thiết tiềm ẩn rủi ro gián đoạn dịch vụ.

12.7. Case Study: KBM

Phần này sẽ áp dụng các lý thuyết về tự động hóa và triển khai vào hệ thống LMS KBM thực tế, đánh giá chi tiết tình hình hiện tại và đề xuất các giải pháp nâng cấp phù hợp với điều kiện của nhà trường.

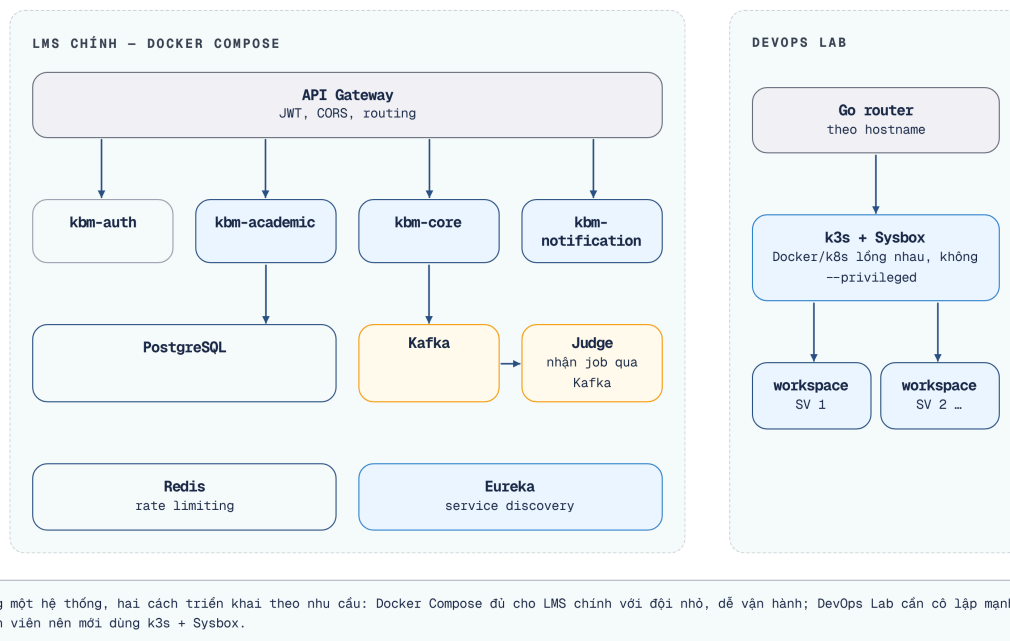
12.7.1. Hiện trạng

KBM hiện có hai kiểu triển khai song song ở mức khái quát:

1. **LMS chính:** các service Java/Spring Boot, database, Kafka, Gateway và frontend triển khai bằng Docker Compose trên hạ tầng nhỏ. Đây là lựa chọn thực dụng cho đội ngũ phát triển quy mô nhỏ, đảm bảo tính dễ vận hành mà không đòi hỏi thiết lập cụm (cluster) phức tạp.
2. **DevOps Lab:** phần thực hành Docker/Kubernetes dùng Go, k3s và Sysbox để tạo môi trường lab cô lập. Đây là nhu cầu đặc thù của miền nghiệp vụ: sinh viên cần chạy container/lab riêng trong môi trường an toàn.

Không cần public domain, IP hay sơ đồ máy chủ cụ thể; điều quan trọng với sách là **sự đánh đổi trong triển khai**: cùng một hệ thống có thể dùng Docker Compose cho phần LMS chính và k3s/Sysbox cho bounded context cần isolation.

Bài toán ở DevOps Lab là sinh viên cần chạy Docker/Kubernetes *bên trong* container thực hành. Cách làm phổ biến — Docker-in-Docker (DinD) — đòi hỏi cấp cờ `--privileged`, tức là mở gần như toàn quyền của host cho container, một rủi ro không chấp nhận được trong môi trường nhiều người dùng. Sysbox giải quyết đúng chỗ đánh đổi này: nó cho phép chạy Docker/k8s lồng nhau mà không cần đặc quyền, nhờ tận dụng user namespace và ảo hóa một phần `procfs` / `sysfs`.



Hình 12.8: Kiến trúc triển khai của KBM ở mức khái quát

12.7.2. Phân tích theo mức độ trưởng thành triển khai

Để đánh giá năng lực triển khai của KBM, chúng ta có thể đối chiếu hiện trạng hệ thống với các tiêu chí trưởng thành chuẩn mực, qua đó nhận diện những lỗ hổng cần sớm khắc phục.

Khía cạnh	Hiện trạng	Mức trưởng thành	Nhận xét
Containerization	Docker images cho tất cả services	● Tốt	Multi-stage builds, container registry (GitLab)
Orchestration	Docker Compose cho LMS chính; k3s/ Sysbox cho DevOps Lab	● Phù hợp theo ngữ cảnh	Không ép một nền tảng cho mọi phần
CI/CD	GitLab CI đã phủ các phần chính; một số phần còn build + push thủ công	● Chưa đồng đều	Phần deploy thủ công còn lại vẫn tạo rủi ro lệch phiên bản (version drift)
Chiến lược triển khai	All-at-once (stop → deploy → start)	● Có downtime	Sinh viên bị gián đoạn khi deploy
IaC	Docker Compose files version controlled	● IaC cơ bản	Tái tạo được nhưng thiếu tự động hóa
Quản lý cấu hình	application.yml trong container	● Mới tách một phần	Một số cấu hình còn gán cứng, chưa tách hoàn toàn ra ngoài
Cô lập lab	k3s + Sysbox cho lab pods	● Đúng bài toán	Cần resource quota, RBAC, NetworkPolicy rõ ràng

Bảng 12.3: Phân tích mức độ trưởng thành triển khai của KBM

12.7.3. Phân tích bối cảnh nghiệp vụ

KBM phục vụ sinh viên, do đó các khung thời gian triển khai (**deployment windows**) có ý nghĩa vô cùng quan trọng:

Khi đã có hạ tầng cho nhiều chiến lược (Giai đoạn 3 bên dưới), có thể chọn theo thời điểm. Vào các khung giờ nhạy cảm như **trước hoặc sau kỳ thi** (rủi ro cao), chiến thuật **Blue/Green** là lựa chọn an toàn nhất để đảm bảo có thể lật ngược tình thế (rollback) ngay lập tức nếu xảy ra sự cố. Trong những ngày **giữa tuần**, tải trọng trung bình cho phép áp dụng **Rolling Update** một cách an toàn. Các đợt bảo trì lớn vào kỳ **nghỉ hè** thường không chịu áp lực downtime, nên cách triển khai toàn bộ cùng lúc (**All-at-once**) là hoàn toàn chấp nhận được. Cuối cùng, với những bản vá khẩn cấp (**Emergency hotfix**), nên đi qua một đợt **Canary** ngắn (nếu hạ tầng cho phép) để thử nghiệm thận trọng trên một nhóm nhỏ trước khi triển khai diện rộng trên toàn hệ thống.

❗ CI/CD chưa đồng đều giữa các phân hệ

KBM đã container hóa và vận hành GitLab CI cho các phần chính (§12.4), nhưng một số phần vẫn **deploy thủ công**: lập trình viên build image trên máy cá nhân, đẩy lên registry, sau đó truy cập từ xa (SSH) vào hạ tầng và thực thi lệnh triển khai. Ở những phần này, kiểm thử và chuyển giao chưa được tự động hóa.

Thực trạng này khiến kiểm thử tự động trước khi deploy chưa phù hợp, cơ chế rollback chưa nhất quán, và mức độ CI/CD giữa LMS chính với DevOps Lab chênh lệch. Hệ quả là khi có lỗi trên production: (1) lập trình viên phát hiện qua báo cáo của người dùng, (2) quy trình sửa mã nguồn, đóng gói, đẩy lên registry và triển khai lại được thực hiện hoàn toàn thủ công, (3) nếu sửa sai, quy trình này lại lặp lại một cách kém hiệu quả. Điều này làm tăng MTTR (Mean Time To Recovery) và gây ra rủi ro nghiêm trọng về việc deploy nhầm phiên bản.

Để giải quyết tận gốc vấn đề, lộ trình chuyển đổi cần được triển khai theo các giai đoạn tăng dần nhằm từng bước hiện đại hóa quy trình:

Giai đoạn 1 — Phủ đều CI/CD (nỗ lực thấp, tác động cao) – Mở rộng quy trình GitLab CI hiện có sang toàn bộ services, tự động chạy kiểm thử và build Docker image sau mỗi lần đẩy mã ở mọi phân hệ. Mục tiêu là không còn đoạn mã nào chưa qua kiểm thử lên được môi trường thực tế, và mọi phiên bản đều truy vết được.

Giai đoạn 2 — Triển khai tự động (nỗ lực trung bình) – Thiết lập triển khai tự động lên staging, kèm smoke test và một bước xét duyệt thủ công trước khi lên production. Health check cho mỗi dịch vụ cho phép triển khai chỉ với một nút bấm, đồng thời lưu lại lịch sử kiểm toán mà không ai phải SSH vào máy chủ.

Giai đoạn 3 — Triển khai không gián đoạn (zero-downtime) (nỗ lực trung bình-cao) – Chuyển sang rolling update với nhiều phiên bản chạy song song, hoặc Blue/Green cho những thời điểm tải cao như kỳ thi, để sinh viên không gặp gián đoạn khi hệ thống cập nhật.

Giai đoạn 4 — Gia cố Kubernetes/cluster (nỗ lực cao) – Chỉ nên thực hiện khi nhu cầu tải thực sự vượt quá khả năng của Docker Compose. Kubernetes mang lại khả năng tự phục hồi, nhưng áp dụng khi chưa cần thiết sẽ đòi hỏi một khối lượng vận hành không tương xứng.

⚠️ Rủi ro trong quá trình triển khai

Việc triển khai microservices phức tạp hơn rất nhiều so với kiến trúc nguyên khối. Một quyết định sai lầm trong quá trình này có thể làm đình trệ toàn bộ hệ thống hoặc gây ra gián đoạn kéo dài.

1. **Triển khai vào chiều Thứ Sáu:** Khi đội ngũ phát hành tính năng mới vào cuối tuần, các lỗi phát sinh thường không được phát hiện do thiếu nhân sự trực chiến. Hậu quả là thời gian gián đoạn có thể kéo dài đến đầu tuần sau. *Phòng tránh:* ưu tiên triển khai vào các buổi sáng giữa tuần khi đội ngũ sẵn sàng xử lý sự cố. Với hệ thống KBM, cần tuyệt đối tránh triển khai trước hoặc trong các đợt thi hoặc buổi thực hành.
2. **Dùng Kubernetes cho hệ thống 3 services:** Tâm lý “Netflix dùng Kubernetes, chúng ta cũng nên” sẽ dẫn đến gánh nặng quản trị phức tạp (cluster management, RBAC, networking) cho một đội ngũ nhỏ. *Phòng tránh:* Docker Compose là đủ cho các hệ thống nhỏ. Việc chuyển đổi sang Kubernetes chỉ thực sự cần thiết khi có nhu cầu chạy trên nhiều máy chủ, yêu cầu mức độ cách ly cao hoặc tự động mở rộng.
3. **Build image khác nhau cho mỗi môi trường:** Việc build riêng cho môi trường phát triển, staging và production sẽ tạo ra các artifact khác nhau. Hậu quả là ứng dụng hoạt động tốt trên staging nhưng lại gặp lỗi trên production. *Phòng tránh:* biên dịch một lần, triển khai mọi nơi (build once, deploy everywhere) — đưa toàn bộ cấu hình phụ thuộc môi trường ra ngoài qua biến môi trường.
4. **Thiếu kế hoạch khôi phục (rollback plan):** Khi phiên bản mới gặp lỗi nhưng không có quy trình khôi phục sẵn sàng, việc can thiệp thủ công trong tình huống khẩn cấp dễ gây ra lỗi phát sinh nghiêm trọng. *Phòng tránh:* mọi quy trình triển khai phải đi kèm tài liệu hướng dẫn khôi phục, quy định rõ ràng các bước kỹ thuật để đưa hệ thống về trạng thái ổn định.
5. **Shared database giữa services** — Chương 7 đã phân tích. Nhưng khi deploy: nếu Service A và B chia sẻ database, database migration của A có thể làm hỏng B. *Phòng tránh:* mỗi service sở hữu database schema riêng — migration độc lập.

💡 Interactive Demo

Xem minh họa động tại companion web:

Các chiến lược Deployment (Blue/Green, Canary) – – deployment-strategies.html

Cấu hình bên ngoài (Externalized Config) – – config-server.html

12.8. Tổng kết

Triển khai microservices phức tạp hơn kiến trúc nguyên khối do có nhiều dịch vụ cần được biên dịch, kiểm thử, triển khai và khôi phục một cách độc lập nhưng vẫn phải đảm bảo tính tương thích chéo. **Containerization** (Docker) giải quyết vấn đề “works on my machine” — đóng gói service và các thư viện phụ thuộc thành một image có thể chạy ở mọi nơi. **Docker Compose** cho phép điều phối nhiều service trên hạ tầng nhỏ — phù hợp cho đội ngũ nhỏ-trung và môi trường development/staging.

CI/CD pipeline là nền tảng cốt lõi: code push → auto build → auto test → auto deploy. Thiếu đi CI/CD, việc triển khai microservices trở thành một “nghỉ thức thủ công” — chậm chạp, đầy rủi ro và

không thể tái tạo (reproducible). Ba chiến lược triển khai chính — Rolling, Blue/Green, Canary — cần được lựa chọn theo mức độ rủi ro: các bản vá lỗi nhỏ áp dụng cập nhật cuốn chiếu, thay đổi cấu trúc lớn sử dụng blue/green, trong khi các tính năng mới nên được thử nghiệm qua canary.

Infrastructure as Code đảm bảo hạ tầng tái tạo được và được quản lý phiên bản — file Docker Compose đã là IaC cơ bản. Khi mở rộng, Terraform + Kubernetes + ArgoCD tạo thành một pipeline GitOps hoàn chỉnh — nhưng cần tránh việc thiết kế quá mức (over-engineering): hãy chọn mức orchestration theo từng bounded context.

Phân tích KBM cho thấy containerization tốt (Docker images, registry) và DevOps Lab đã có nhu cầu k3s/Sysbox riêng, nhưng CI/CD chưa phù hợp và mỗi lần deploy vẫn gây gián đoạn (all-at-once). Lộ trình chuyển đổi rõ ràng: CI/CD cơ bản → triển khai tự động → chiến lược zero-downtime → gia cố cluster khi thật sự cần.

Qua 12 chương, chúng ta đã đi từ nền tảng SOA/Microservices (Ch.1-2), qua các mẫu giao tiếp (Ch.3-6), quản lý dữ liệu (Ch.7), hạ tầng (Ch.8-9), chuyển đổi thực tế (Ch.10), observability (Ch.11), đến triển khai (Ch.12). Quá trình chuyển đổi từ monolith đến microservices không phải là một cuộc “chuyển đổi Big Bang” tức thời — mà là **chuỗi quyết định nhỏ, mỗi quyết định mang lại giá trị thực tiễn ngay lập tức** đi kèm với việc thấu hiểu sự đánh đổi (trade-off) của từng mẫu thiết kế.

12.9. Đọc thêm

Sách tham khảo chính:

1. [2a] Chris Richardson, *Microservices Patterns* 1st Ed. — Ch.12: Deploying Microservices — deployment patterns (language-specific, VM, container, Kubernetes, serverless)
2. [4a] Sam Newman, *Building Microservices* — Ch.6: Deployment — CI, build pipelines, CD, Docker, service-to-host mapping và nền tảng điều phối container
3. [3] Ronnie Mitra & Irakli Nadareishvili, *Microservices: Up and Running* — Ch.6-7: Infrastructure Pipeline & Infrastructure — IaC, Terraform, Kubernetes; Ch.10: Releasing — Docker, Helm, ArgoCD; Ch.11: Managing Change — deployment patterns

Sách bổ trợ:

4. [2b] Chris Richardson, *Microservices Patterns* 2nd Ed. — Ch.18-19: Deploying Microservices & on Kubernetes (updated)
5. [4b] Sam Newman, *Monolith to Microservices* — Ch.3: Splitting the Monolith — deployment considerations during migration

Nguồn trực tuyến:

- Docker official docs — docs.docker.com
- Docker Compose documentation — docs.docker.com/compose
- Kubernetes official docs — kubernetes.io/docs
- Martin Fowler, “Continuous Delivery” — martinfowler.com/bliki/ContinuousDelivery.html
- Martin Fowler, “BlueGreenDeployment” — martinfowler.com/bliki/BlueGreenDeployment.html
- Terraform by HashiCorp — terraform.io
- ArgoCD — argo-cd.readthedocs.io (GitOps for Kubernetes)

Bài tập: Xem Phụ lục Bài tập để luyện tập:

12-0 – Ôn tập: Các chiến lược triển khai ([L1])

12-1 – Docker Compose vs Kubernetes — Ở quy mô nào? (Trade-off Analysis [L2])

12-2 – Design: CI/CD Pipeline cho Microservices (System Design [L3])

Tài liệu tham khảo

- Mitra, R. & Nadareishvili, I. (2020). *Microservices: Up and Running*. O'Reilly Media.